C O D I G O F U E N T E

Mi Director Financiero PTY

Plataforma de Alta Direccion Financiera y Legal
para Emprendedores Panamenos

Obra:	Programa de Computadora (Software)
Version:	0.1.0
Autora:	Soraya Dayan Gutierrez Perez
Titular:	Soraya Dayan Gutierrez Perez (Persona Natural)
Fecha:	Marzo 2026
Archivos totales:	112 archivos fuente
Lineas de codigo:	~46,917 lineas

Documento presentado ante la Direccion Nacional de Derecho de Autor (DNDA) del Ministerio de Comercio e Industrias de la Republica de Panama, para efectos del registro de derechos de autor sobre obra literaria tipo programa de computadora.

1. Informacion de la Obra

Datos Generales del Programa	
Nombre del programa	Mi Director Financiero PTY
Version	0.1.0
Tipo de obra	Programa de computadora (software) - Aplicacion web
Fecha de creacion	18 de febrero de 2026
Pais de origen	Panama
Estado	Obra inedita (en desarrollo / acceso restringido)

Lenguajes y Tecnologias	
Lenguaje principal (Frontend)	TypeScript 5.7 / JavaScript ES2017+
Lenguaje secundario (Backend)	Python 3.11+
Framework Frontend	Next.js 15.1.0 + React 19
Framework Backend	FastAPI 0.115+
Estilos	Tailwind CSS 4.0
Base de datos	PostgreSQL (Supabase) con Row Level Security
Inteligencia Artificial	GPT-4o (OpenAI) para asistente conversacional
Hosting Frontend	Vercel
Hosting Backend	Railway

Metricas del Codigo	
Total archivos fuente	112 archivos
Total lineas de codigo	~46,917 lineas
Archivos TypeScript/TSX	~70 archivos (~38,000 lineas)
Archivos Python	~42 archivos (~8,900 lineas)
Dependencias Frontend	7 librerias principales (next, react, recharts, supabase, lucide-react, tailwindcss, vitest)
Dependencias Backend	fastapi, supabase-py, openai, pydantic, httpx

2. Estructura del Proyecto (Arbol de Directorios)

Se presenta a continuacion la estructura completa de archivos fuente del proyecto, excluyendo dependencias de terceros (node_modules), archivos compilados (.next) y archivos de entorno (.env).

```
mi-director-financiero-pty/
|
+-- frontend/                          # Aplicacion Next.js (TypeScript)
|   +-- package.json                   # Dependencias y scripts
|   +-- next.config.ts                 # Configuracion Next.js + API proxy
|   +-- tsconfig.json                  # Configuracion TypeScript
|   +-- postcss.config.mjs             # PostCSS + Tailwind
|   |
|   +-- src/
|       +-- app/
|       |   +-- layout.tsx             # Layout raiz (fuentes, metadata)
|       |   +-- page.tsx               # Redirect a /dashboard
|       |   +-- globals.css            # Estilos globales + animaciones
|       |   +-- dashboard/
|       |       +-- page.tsx           # Dashboard principal (~2,093 lineas)
|       |
|       +-- components/
|       |   +-- MidfLogo.tsx           # Logo SVG (brujula + $)
|       |   +-- AuthProvider.tsx       # Contexto autenticacion Supabase
|       |   +-- AccessGate.tsx         # Gate con clave de acceso
|       |   +-- AlertsSidebar.tsx      # Panel lateral de alertas
|       |   +-- SmartTooltip.tsx       # Glosario inteligente (100+ terminos)
|       |   +-- PayrollEngine.tsx      # Calculadora de nomina Panama
|       |   +-- LabPrecios.tsx         # Lab de Precios (costeo unitario)
|       |   +-- EspejoDGI.tsx          # Espejo de Formularios DGI
|       |   +-- DataEntryWizard.tsx    # Wizard datos financieros
|       |   +-- NaturalLanguageInput.tsx # Input con IA
|       |   +-- BottomNavBar.tsx       # Navegacion inferior mobile
|       |   +-- AttendanceModal.tsx    # Modal de asistencia
|       |   +-- LibroContable.tsx      # Libro Diario/Mayor
|       |   +-- ChartOfAccounts.tsx    # Plan de Cuentas Panama
|       |   +-- LibroInventarios.tsx   # Libro de Inventarios (contable)
|       |   +-- BalanceComprobacion.tsx # Balance de Comprobacion
|       |   +-- ValoracionTab.tsx      # Valoracion de empresa
|       |   +-- Ley186Monitor.tsx      # Monitor Ley 186 S.E.P.
|       |   +-- FlashDataEntry.tsx     # Diagnostico Flash
|       |   +-- BudgetModule.tsx       # Modulo de Presupuesto
|       |   |
|       |   +-- declaraciones/         # Formularios DGI
|       |   |   +-- F2V10BorradorScreen.tsx  # Form F2 V10 (S.E.P.)
|       |   |   +-- Form20Screen.tsx         # Form 20 (Pagos Terceros)
|       |   |   +-- PlanillaCSSScreen.tsx    # Planilla CSS
|       |   |   +-- ReconciliacionIngresosF2.tsx
|       |   |
|       |   +-- espejo-dgi/            # Sub-pantallas DGI
|       |   |   +-- Form20Screen.tsx
|       |   |   +-- PlanillaCSSScreen.tsx
|       |   |
|       |   +-- inventario/            # Modulo Mi Inventario
|       |       +-- MiInventario.tsx   # CRUD + Dashboard (~1,100 lineas)
|       |
|       +-- lib/
|       |   +-- calculations.ts        # Motor financiero (cascada, ratios)
|       |   +-- alerts.ts              # Motor de alertas centralizado
```

```
|       |   +-- unit-conversions.ts    # Conversiones peso/volumen/conteo
|       |   +-- inventario-types.ts    # Tipos + CRUD inventario
|       |   +-- rrhh-types.ts          # Tipos + CRUD recursos humanos
|       |   +-- dgi-mappings.ts        # Mapeo formularios DGI
|       |   +-- currency.ts            # Formato Balboas (B/.)
|       |   +-- sounds.ts             # Sonidos Web Audio API
|       |   +-- api.ts                 # Cliente API (JWT + demo mode)
|       |   +-- supabase/
|       |       +-- client.ts          # Cliente Supabase browser
|       |
|       +-- services/
|           +-- declaraciones/
|               +-- f2v10Draft.service.ts  # Servicio borrador F2 V10
|
+-- backend/                           # API FastAPI (Python)
|   +-- main.py                        # Entry point + routers
|   +-- app/
|   |   +-- auth.py                    # Autenticacion JWT Supabase
|   |   +-- database.py               # Conexion Supabase
|   |   +-- models.py                 # Modelos Pydantic
|   |   +-- middleware.py              # Rate limiting + CORS
|   |   +-- routers/
|   |   |   +-- financial.py          # CRUD + Motor Financiero
|   |   |   +-- societies.py          # CRUD Sociedades
|   |   |   +-- payroll.py            # Nomina
|   |   |   +-- accounting.py         # Contabilidad (Libros)
|   |   |   +-- nlp.py                # Procesamiento Lenguaje Natural
|   |   |   +-- ai_agents.py          # Agentes IA
|   |   |   +-- audit.py              # Auditoria
|   |   |   +-- reports.py            # Reportes PDF
|   |   |   +-- budget.py             # Presupuesto
|   |   +-- engines/
|   |       +-- financial_engine.py    # Cascada, ratios, valoracion
|   |       +-- panama_payroll.py      # Nomina Panama (Ley 462)
|   |       +-- accounting_engine.py   # Motor contable
|   |       +-- nlp_engine.py          # Motor lenguaje natural
|   |       +-- multiperiod_engine.py  # Tendencias multi-periodo
|   |       +-- pdf_generator.py       # Generador PDF
|   |       +-- csv_exporter.py        # Exportador CSV
|   |       +-- panama_calendar.py     # Calendario fiscal Panama
|   |       +-- email_sender.py        # Envio de correos
|   |       +-- account_mapping.py     # Mapeo cuentas contables
|   |       +-- scheduled_tasks.py     # Tareas programadas
|   +-- tests/
|       +-- conftest.py
|       +-- engines/
|           +-- test_financial_engine.py
|           +-- test_panama_payroll.py
|           +-- test_accounting_engine.py
|           +-- test_nlp_engine.py
|           +-- test_multiperiod_engine.py
|           +-- test_panama_calendar.py
|           +-- test_csv_exporter.py
|           +-- test_pdf_generator.py
|           +-- test_auto_journal.py
|           +-- test_account_mapping.py
|           +-- test_pdf_executive_reports.py
|
+-- docs/
    +-- dnda/                          # Documentos DNDA
        +-- DNDA_MemoriaDescriptiva.html
        +-- DNDA_CodigoFuente_MiDirectorFinancieroPTY.html
```

3. Codigo Fuente Representativo

Se incluyen a continuacion los archivos fuente mas representativos del programa, seleccionados para demostrar la originalidad, complejidad y alcance funcional de la obra. Los archivos se presentan en orden logico: configuracion, punto de entrada, nucleo de negocio, componentes de interfaz y backend.

3.1 Configuracion del Proyecto

frontend/package.json JSON

```
{
  "name": "mi-director-financiero-pty",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "test": "vitest run"
  },
  "dependencies": {
    "@supabase/ssr": "^0.5.0",
    "@supabase/supabase-js": "^2.47.0",
    "lucide-react": "^0.468.0",
    "next": "^15.1.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "recharts": "^2.15.0"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4.0.0",
    "@types/node": "^22.0.0",
    "@types/react": "^19.0.0",
    "@types/react-dom": "^19.0.0",
    "eslint": "^9.0.0",
    "eslint-config-next": "^15.1.0",
    "postcss": "^8.5.0",
    "tailwindcss": "^4.0.0",
    "typescript": "^5.7.0",
    "vitest": "^4.0.18"
  }
}
```

frontend/next.config.ts TypeScript

```
import type { NextConfig } from "next";

const backendUrl = process.env.NEXT_PUBLIC_API_URL || "http://localhost:8000";

const nextConfig: NextConfig = {
  // Proxy API calls to FastAPI backend
  async rewrites() {
    return [
      {
        source: "/api/:path*",
```

```
        destination: `${backendUrl}/api/:path*`,
      },
    ];
  },
};

export default nextConfig;
```

frontend/tsconfig.json JSON

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules", "vitest.config.ts"]
}
```

3.2 Punto de Entrada y Layout

frontend/src/app/layout.tsx TypeScript/React

```
import type { Metadata } from "next";
import { Montserrat, Playfair_Display } from "next/font/google";
import "./globals.css";
import { AuthProvider } from "@/components/AuthProvider";
import { AccessGate } from "@/components/AccessGate";

const montserrat = Montserrat({
  subsets: ["latin"],
  variable: "--font-body",
  display: "swap",
});

const playfair = Playfair_Display({
  subsets: ["latin"],
  variable: "--font-heading",
  display: "swap",
});

export const metadata: Metadata = {
  title: "Mi Director Financiero PTY | Tu Aliado Estrategico",
  description:
```

```
    "Plataforma de Alta Direccion financiera y legal para emprendedores panamenos.",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html lang="es" suppressHydrationWarning>
      <body
        className={`${montserrat.variable} ${playfair.variable} font-sans`}
        suppressHydrationWarning
      >
        <AccessGate>
          <AuthProvider>{children}</AuthProvider>
        </AccessGate>
      </body>
    </html>
  );
}
```

frontend/src/app/page.tsx TypeScript/React

```
import { redirect } from "next/navigation";

export default function Home() {
  redirect("/dashboard");
}
```

frontend/src/app/globals.css CSS

```
@import "tailwindcss";

/* MI DIRECTOR FINANCIERO PTY -- Light Theme */

body {
  font-family: var(--font-body), 'Montserrat', system-ui, -apple-system, sans-serif;
}

h1, h2, h3, .font-heading {
  font-family: var(--font-heading), 'Playfair Display', Georgia, serif;
}

@keyframes shimmer {
  0% { background-position: -200% center; }
  100% { background-position: 200% center; }
}

@keyframes veredicto-pulse {
  0%, 100% { box-shadow: 0 0 0 0 rgba(197, 160, 89, 0.4); }
  50% { box-shadow: 0 0 20px 4px rgba(197, 160, 89, 0.15); }
}
```

3.3 Nucleo de Logica Financiera

frontend/src/lib/calculations.ts TypeScript

```typescript
/**
 * Funciones de calculo financiero client-side para Mi Director Financiero PTY.
 * Usadas por SimuladorEstrategico, LabPrecios y ValoracionTab.
 */

// TIPOS

export interface FinancialRecord {
  revenue: number;
  cogs: number;
  opex_rent: number;
  opex_payroll: number;
  opex_other: number;
  depreciation: number;
  interest_expense: number;
  tax_expense: number;
  cash_balance: number;
  accounts_receivable: number;
  inventory: number;
  accounts_payable: number;
  bank_debt: number;
}

export interface CascadaResult {
  revenue: number;
  cogs: number;
  gross_profit: number;
  total_opex: number;
  ebitda: number;
  ebit: number;
  ebt: number;
  net_income: number;
  gross_margin_pct: number;
  ebitda_margin_pct: number;
  net_margin_pct: number;
}

export interface PricingResult {
  costo_materiales: number;
  costo_mano_obra: number;
  costo_fijo_unitario: number;
  costo_total_unitario: number;
  precio_sin_itbms: number;
  itbms: number;
  precio_final: number;
  ganancia_por_unidad: number;
}

export type UnitCategory = "weight" | "volume" | "count";

export interface UnitOfMeasure {
  id: string;
  label: string;
  category: UnitCategory;
  conversionFactor: number;
  isCustom?: boolean;
}
```

```
export interface Ingredient {
  id: string;
  nombre: string;
  costoAdquisicion: number;
  cantidadCompra?: number;
  unidadCompraId: string;
  cantidadUtilizada: number;
  unidadUsoId?: string;
  costo: number;
  modoSimple?: boolean;
}

// CASCADA (P&L)

export function computeCascada(r: FinancialRecord): CascadaResult {
  const gross_profit = r.revenue - r.cogs;
  const total_opex = r.opex_rent + r.opex_payroll + r.opex_other;
  const ebitda = gross_profit - total_opex;
  const ebit = ebitda - r.depreciation;
  const ebt = ebit - r.interest_expense;
  const net_income = ebt - r.tax_expense;

  return {
    revenue: r.revenue,
    cogs: r.cogs,
    gross_profit,
    total_opex,
    ebitda,
    ebit,
    ebt,
    net_income,
    gross_margin_pct: r.revenue > 0 ? (gross_profit / r.revenue) * 100 : 0,
    ebitda_margin_pct: r.revenue > 0 ? (ebitda / r.revenue) * 100 : 0,
    net_margin_pct: r.revenue > 0 ? (net_income / r.revenue) * 100 : 0,
  };
}

// SIMULADOR ESTRATEGICO

export function computeSimulation(
  base: FinancialRecord,
  priceChangePct: number,
  costChangePct: number,
  volumeChangePct: number
): SimulationResult {
  const original = computeCascada(base);
  const fPrecio = 1 + priceChangePct / 100;
  const fCostos = 1 - costChangePct / 100;
  const fVolumen = 1 + volumeChangePct / 100;

  const simRecord: FinancialRecord = {
    ...base,
    revenue: base.revenue * fPrecio * fVolumen,
    cogs: base.cogs * fVolumen,
    opex_rent: base.opex_rent * fCostos,
    opex_payroll: base.opex_payroll * fCostos,
    opex_other: base.opex_other * fCostos,
  };

  const simulated = computeCascada(simRecord);
  return {
    original,
    simulated,
    delta_ebitda: simulated.ebitda - original.ebitda,
```

```
    delta_margin: simulated.ebitda_margin_pct - original.ebitda_margin_pct,
  };
}

// ESCUDO CONTRA EL MIEDO

export function computeMaxClientLoss(
  priceIncreasePct: number,
  revenue: number,
  cogs: number
): number {
  if (priceIncreasePct <= 0 || revenue <= 0) return 0;
  const mcPct = (revenue - cogs) / revenue;
  const pDelta = priceIncreasePct / 100;
  if (pDelta + mcPct <= 0) return 0;
  return (pDelta / (pDelta + mcPct)) * 100;
}

// CALCULADOR DE PRECIO

export function computePricing(
  ingredientes: Ingredient[],
  salarioMensual: number,
  minutosElaboracion: number,
  opexMensualTotal: number,
  capacidadMensual: number,
  margenDeseadoPct: number,
  comisionPlataformaPct: number = 0
): PricingResult {
  const costo_materiales = ingredientes.reduce((sum, i) => sum + i.costo, 0);
  const costoMinuto = salarioMensual / 11520;
  const costo_mano_obra = costoMinuto * minutosElaboracion;
  const costo_fijo_unitario =
    capacidadMensual > 0 ? opexMensualTotal / capacidadMensual : 0;
  const costo_total_unitario =
    costo_materiales + costo_mano_obra + costo_fijo_unitario;
  const denominador =
    1 - (margenDeseadoPct + comisionPlataformaPct) / 100;

  let precio_sin_itbms = 0;
  let ganancia_por_unidad = 0;
  if (denominador > 0) {
    precio_sin_itbms = costo_total_unitario / denominador;
    ganancia_por_unidad = precio_sin_itbms - costo_total_unitario;
  }
  const itbms = precio_sin_itbms * 0.07;
  const precio_final = precio_sin_itbms + itbms;

  return {
    costo_materiales,
    costo_mano_obra,
    costo_fijo_unitario,
    costo_total_unitario,
    precio_sin_itbms,
    itbms,
    precio_final,
    ganancia_por_unidad,
  };
}

// VALORACION EMPRESA

export function computeValoracion(
  ebitdaMensual: number,
  multiplo: number,
```

```
  esDuenoLocal: boolean = false,
  alquilerVirtual: number = 0,
  valorEdificio: number = 0,
  deudaTotal: number = 0
) {
  const ebitdaAnual = esDuenoLocal
    ? (ebitdaMensual - alquilerVirtual) * 12
    : ebitdaMensual * 12;
  const valorOperativo = Math.max(ebitdaAnual * multiplo, 0);
  const patrimonio = valorOperativo + (esDuenoLocal ? valorEdificio : 0) - deudaTotal;
  return { ebitdaAjustado: ebitdaAnual, valorOperativo, patrimonio };
}

// OXIGENO (Ratios de liquidez)

export function computeOxigeno(r: FinancialRecord) {
  const diasCalle =
    r.revenue > 0 ? (r.accounts_receivable / r.revenue) * 30 : 0;
  const diasInventario =
    r.cogs > 0 ? (r.inventory / r.cogs) * 30 : 0;
  const diasProveedor =
    r.cogs > 0 ? (r.accounts_payable / r.cogs) * 30 : 0;
  const ccc = diasCalle + diasInventario - diasProveedor;
  const totalLiquido = r.cash_balance + r.accounts_receivable;
  const pasivoCorto = r.accounts_payable + r.bank_debt;
  const pruebaAcida = pasivoCorto > 0 ? totalLiquido / pasivoCorto : 0;
  const dineroAtrapado = r.accounts_receivable + r.inventory;
  const cascada = computeCascada(r);
  const coberturaBancaria =
    r.interest_expense > 0 ? cascada.ebitda / r.interest_expense : 10;
  return {
    diasCalle, diasInventario, diasProveedor, ccc,
    pruebaAcida, dineroAtrapado, coberturaBancaria,
    totalLiquido, pasivoCorto,
  };
}
```

frontend/src/lib/unit-conversions.ts TypeScript

```
/**
 * Tabla de equivalencias y funciones de conversion para materiales/ingredientes.
 * Soporta unidades de peso (gramos), volumen (mililitros) y conteo (discretas).
 */

import type { UnitOfMeasure, UnitCategory, Ingredient } from "./calculations";

export const BUILTIN_UNITS: UnitOfMeasure[] = [
  // Peso - conversion a gramos
  { id: "quintal", label: "Quintal", category: "weight", conversionFactor: 45_359 },
  { id: "kg", label: "Kilogramo", category: "weight", conversionFactor: 1_000 },
  { id: "lb", label: "Libra", category: "weight", conversionFactor: 453.59 },
  { id: "g", label: "Gramo", category: "weight", conversionFactor: 1 },
  // Volumen - conversion a mililitros
  { id: "galon", label: "Galon", category: "volume", conversionFactor: 3_785 },
  { id: "litro", label: "Litro", category: "volume", conversionFactor: 1_000 },
  { id: "ml", label: "Mililitro", category: "volume", conversionFactor: 1 },
  // Discretas / Conteo
  { id: "unidad", label: "Unidad", category: "count", conversionFactor: 1 },
  { id: "caja", label: "Caja", category: "count", conversionFactor: 1 },
  { id: "paquete", label: "Paquete", category: "count", conversionFactor: 1 },
  { id: "metro", label: "Metro", category: "count", conversionFactor: 1 },
  { id: "pie", label: "Pie", category: "count", conversionFactor: 1 },
];
```

```
export function findUnit(
  unitId: string,
  customUnits: UnitOfMeasure[] = []
): UnitOfMeasure | undefined {
  return (
    BUILTIN_UNITS.find((u) => u.id === unitId) ??
    customUnits.find((u) => u.id === unitId)
  );
}

export function computeIngredientCost(
  costoAdquisicion: number,
  unitId: string,
  cantidadUtilizada: number,
  customUnits: UnitOfMeasure[] = [],
  cantidadCompra: number = 1,
  unidadUsoId?: string,
): number {
  if (costoAdquisicion <= 0 || cantidadUtilizada <= 0) return 0;
  const qty = cantidadCompra > 0 ? cantidadCompra : 1;
  const unit = findUnit(unitId, customUnits);
  if (!unit || unit.conversionFactor <= 0) return 0;
  const costoPorBaseUnit = costoAdquisicion / (qty * unit.conversionFactor);
  let usageInBaseUnits = cantidadUtilizada;
  if (unidadUsoId) {
    const usageUnit = findUnit(unidadUsoId, customUnits);
    if (usageUnit && usageUnit.conversionFactor > 0) {
      usageInBaseUnits = cantidadUtilizada * usageUnit.conversionFactor;
    }
  }
  return costoPorBaseUnit * usageInBaseUnits;
}
```

frontend/src/lib/currency.ts TypeScript

```
/**
 * Currency formatting utility for Panama (Balboa / USD).
 * The Balboa (B/.) is pegged 1:1 to USD.
 */

export function formatBalboas(amount: number, options?: FormatOptions): string {
  const {
    decimals = true,
    decimalPlaces = 2,
    showSign = false,
    compact = false,
    useDollarSign = false,
  } = options ?? {};

  const prefix = useDollarSign ? "$" : "B/.";

  if (compact) {
    if (Math.abs(amount) >= 1_000_000) {
      return `${prefix} ${(amount / 1_000_000).toFixed(1)}M`;
    }
    if (Math.abs(amount) >= 10_000) {
      return `${prefix} ${(amount / 1_000).toFixed(1)}K`;
    }
  }

  const formatted = amount.toLocaleString("es-PA", {
    minimumFractionDigits: decimals ? decimalPlaces : 0,
```

```
    maximumFractionDigits: decimals ? decimalPlaces : 0,
  });

  const sign = showSign && amount > 0 ? "+" : "";
  return `${prefix} ${sign}${formatted}`;
}

export function parseBalboas(input: string): number {
  const cleaned = input
    .replace(/B\/\.\s?/, "")
    .replace(/\$/g, "")
    .replace(/,/g, "");
  return parseFloat(cleaned) || 0;
}
```

3.4 Motor de Alertas Centralizado

frontend/src/lib/alerts.ts TypeScript

```
/**
 * Motor de Alertas Centralizado -- Mi Director Financiero PTY
 * Genera alertas contextuales basadas en datos financieros,
 * legales y laborales del negocio.
 */

import type { FinancialRecord } from "@/lib/calculations";
import { computeCascada, computeOxigeno } from "@/lib/calculations";

export type AlertPriority = "red" | "orange" | "yellow" | "green";
export type AlertCategory = "dgi" | "capital_humano" | "liquidez"
  | "rentabilidad" | "legal" | "inventario";

export interface StrategicAlert {
  id: string;
  priority: AlertPriority;
  category: AlertCategory;
  title: string;
  message: string;
  emoji: string;
  promptHint: string;
}

export function computeAlerts(record: FinancialRecord): StrategicAlert[] {
  const alerts: StrategicAlert[] = [];
  const cascada = computeCascada(record);
  const ox = computeOxigeno(record);

  // RENTABILIDAD
  if (cascada.net_income < 0) {
    alerts.push({
      id: "net_loss", priority: "red", category: "rentabilidad",
      title: "Perdida Neta",
      message: `Tu negocio esta perdiendo $${Math.abs(cascada.net_income).toLocaleString("es-PA")} al mes.`,
      emoji: "🚨", promptHint: "Tu negocio esta en numeros rojos",
    });
  }
  if (cascada.ebitda < 0) {
    alerts.push({
      id: "ebitda_negative", priority: "red", category: "rentabilidad",
      title: "EBITDA Negativo",
      message: `Tu operacion no genera caja. EBITDA: -$${Math.abs(cascada.ebitda).toLocaleString("es-PA")}.`,
      emoji: "💀", promptHint: "Tu operacion no genera caja suficiente",
    });
  }

  // LIQUIDEZ
  if (ox.pruebaAcida < 0.5) {
    alerts.push({
      id: "acid_critical", priority: "red", category: "liquidez",
      title: "Asfixia de Liquidez",
      message: `Prueba Acida: ${ox.pruebaAcida.toFixed(2)}x. No puedes cubrir deudas.`,
      emoji: "🆘", promptHint: "No tienes liquidez para pagar deudas",
    });
  }

  // NOMINA (Factor 1.36x Ley 462/2025)
```

```
  const FACTOR_COSTO_REAL = 1.36;
  const grossProfit = record.revenue - record.cogs;
  const costoRealNomina = record.opex_payroll * FACTOR_COSTO_REAL;
  const payrollRatio = grossProfit > 0 ? (costoRealNomina / grossProfit) * 100 : 0;
  if (payrollRatio > 30 && record.opex_payroll > 0) {
    alerts.push({
      id: "payroll_high",
      priority: payrollRatio > 45 ? "red" : "orange",
      category: "capital_humano",
      title: "Sobrecosto Laboral",
      message: `La nomina consume ${payrollRatio.toFixed(1)}% de tu utilidad bruta.`,
      emoji: "⚠️", promptHint: "La planilla supero el 30% de tu U.B.",
    });
  }

  // FISCAL / DGI
  const annualRevenue = record.revenue * 12;
  if (annualRevenue >= 42000) {
    alerts.push({
      id: "itbms_obligatorio", priority: "orange", category: "dgi",
      title: "ITBMS Obligatorio",
      message: `Ventas anuales proyectadas: $${annualRevenue.toLocaleString("es-PA")}. Debes cobrar 7% ITBMS.`,
      emoji: "📋", promptHint: "ITBMS obligatorio",
    });
  }

  // ... 20+ reglas adicionales de alerta (compliance, Ley 186, feriados, etc.)

  if (alerts.length === 0) {
    alerts.push({
      id: "all_ok", priority: "green", category: "rentabilidad",
      title: "Empresa Saludable",
      message: "Todos los indicadores estan en rango optimo.",
      emoji: "✅", promptHint: "Tu empresa esta saludable",
    });
  }

  alerts.sort((a, b) => priorityOrder[a.priority] - priorityOrder[b.priority]);
  return alerts;
}

// Compliance, Payroll, Ley 186 alerts... (funciones adicionales omitidas por brevedad)
```

3.5 Modulo de Inventario — Tipos y Logica

frontend/src/lib/inventario-types.ts TypeScript

```
/**
 * Tipos, interfaces y utilidades para el modulo Mi Inventario.
 * Keys localStorage:
 *   midf_inventario_catalogo     -> InventarioItem[]
 *   midf_inventario_movimientos  -> MovimientoInventario[]
 *   midf_inventario_categorias   -> string[]
 *   midf_inventario_config       -> InventarioConfig
 */

export const CATEGORIAS_DEFAULT: string[] = [
  "Materia Prima", "Empaque", "Producto Terminado",
  "Insumo", "Repuesto", "Otro",
];

export const SKU_PREFIXES: Record<string, string> = {
  "Materia Prima": "MP", "Empaque": "EMP",
  "Producto Terminado": "PT", "Insumo": "INS",
  "Repuesto": "REP", "Otro": "OTR",
};

export interface InventarioItem {
  id: string;
  sku: string;
  nombre: string;
  descripcion?: string;
  categoria: string;
  cantidad_actual: number;
  unidad_medida: string;
  costo_unitario: number;
  precio_venta: number;
  punto_reorden: number;
  modo_calculo_rop: "manual" | "automatico";
  lead_time_dias?: number;
  consumo_diario_promedio?: number;
  safety_stock: number;
  proveedor_principal?: string;
  estado: "activo" | "descontinuado";
  fecha_creacion: string;
  ultima_actualizacion: string;
  societyId: string;
}

export type TipoMovimiento =
  | "entrada_compra" | "entrada_produccion"
  | "entrada_devolucion_cliente" | "entrada_ajuste_positivo"
  | "salida_venta" | "salida_consumo_produccion"
  | "salida_merma_perdida" | "salida_devolucion_proveedor"
  | "salida_ajuste_negativo";

export interface MovimientoInventario {
  id: string;
  fecha: string;
  tipo: TipoMovimiento;
  item_id: string;
  item_sku: string;
  item_nombre: string;
  cantidad: number;
```

```
  costo_unitario_momento: number;
  referencia?: string;
  cantidad_resultante: number;
  societyId: string;
}

// Calculo ROP automatico
export function calcularROPAutomatico(
  leadTimeDias: number,
  consumoDiarioPromedio: number,
  safetyStock: number,
): number {
  return Math.ceil((leadTimeDias * consumoDiarioPromedio) + safetyStock);
}

// Costo promedio ponderado al recibir compra
export function calcularCostoPromedioPonderado(
  cantidadExistente: number,
  costoActual: number,
  cantidadCompra: number,
  costoCompra: number,
): number {
  const totalCantidad = cantidadExistente + cantidadCompra;
  if (totalCantidad <= 0) return costoCompra;
  return Math.round(
    ((cantidadExistente * costoActual) + (cantidadCompra * costoCompra)) / totalCantidad
  * 100) / 100;
}

// SKU auto-generation
export function generarSKU(categoria: string, items: InventarioItem[]): string {
  const prefix = SKU_PREFIXES[categoria] || "OTR";
  const existingNumbers = items
    .filter((i) => i.sku.startsWith(prefix + "-"))
    .map((i) => parseInt(i.sku.split("-").pop() || "0", 10) || 0);
  const nextNum = existingNumbers.length > 0 ? Math.max(...existingNumbers) + 1 : 1;
  return `${prefix}-${String(nextNum).padStart(3, "0")}`;
}

// CRUD localStorage
export function loadCatalogo(): InventarioItem[] {
  if (typeof window === "undefined") return [];
  try {
    const raw = localStorage.getItem("midf_inventario_catalogo");
    return raw ? JSON.parse(raw) : [];
  } catch { return []; }
}

export function saveCatalogo(items: InventarioItem[]): void {
  localStorage.setItem("midf_inventario_catalogo", JSON.stringify(items));
}

// Alertas de inventario
export function computeInventarioAlerts(
  items: InventarioItem[],
  movimientos: MovimientoInventario[],
  config: InventarioConfig,
): StrategicAlert[] {
  const alerts: StrategicAlert[] = [];
  const activos = items.filter((i) => i.estado === "activo");
  for (const item of activos) {
    if (item.cantidad_actual === 0) {
      alerts.push({
        id: `inv-agotado-${item.id}`, priority: "red",
        category: "inventario", title: "Producto agotado",
```

```
        message: `${item.nombre} (SKU: ${item.sku}) esta AGOTADO.`,
        emoji: "🚨", promptHint: `${item.nombre} esta agotado`,
      });
    } else if (item.punto_reorden > 0 && item.cantidad_actual <= item.punto_reorden) {
      alerts.push({
        id: `inv-reorden-${item.id}`, priority: "orange",
        category: "inventario", title: "Punto de reorden alcanzado",
        message: `${item.nombre}: Stock ${item.cantidad_actual}/${item.punto_reorden}.`,
        emoji: "📦", promptHint: `${item.nombre} necesita reorden`,
      });
    }
  }
  // ... alertas de items sin movimiento y valor inmovilizado
  return alerts;
}
```

3.6 Autenticacion y Seguridad

frontend/src/components/AuthProvider.tsx TypeScript/React

```
"use client";
/**
 * AuthProvider -- Contexto React para estado de autenticacion.
 * Provee user, session, loading, signOut, isDemoMode a toda la app.
 */
import { createContext, useContext, useEffect, useState } from "react";
import { createClient, isSupabaseConfigured } from "@/lib/supabase/client";
import type { User, Session } from "@supabase/supabase-js";

type AuthContextType = {
  user: User | null;
  session: Session | null;
  loading: boolean;
  signOut: () => Promise<void>;
  isDemoMode: boolean;
};

const AuthContext = createContext<AuthContextType>({
  user: null, session: null, loading: true,
  signOut: async () => {}, isDemoMode: false,
});

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [session, setSession] = useState<Session | null>(null);
  const [loading, setLoading] = useState(true);
  const supabase = createClient();
  const isDemoMode = process.env.NEXT_PUBLIC_DEMO_MODE === "true";
  const configured = isSupabaseConfigured();

  useEffect(() => {
    if (!configured) { setLoading(false); return; }
    supabase.auth.getSession()
      .then(({ data: { session: s } }) => {
        setSession(s); setUser(s?.user ?? null); setLoading(false);
      }).catch(() => setLoading(false));

    const { data: { subscription } } = supabase.auth.onAuthStateChange((_event, s) => {
      setSession(s); setUser(s?.user ?? null); setLoading(false);
    });
    return () => subscription.unsubscribe();
  }, []);

  const signOut = async () => {
    if (configured) await supabase.auth.signOut();
    window.location.href = "/";
  };

  return (
    <AuthContext.Provider value={{ user, session, loading, signOut, isDemoMode }}>
      {children}
    </AuthContext.Provider>
  );
}

export const useAuth = () => useContext(AuthContext);
```

frontend/src/lib/supabase/client.ts TypeScript

```typescript
/**
 * Supabase Browser Client -- para Client Components ("use client")
 * Usa @supabase/ssr para manejo correcto de cookies en Next.js 15.
 */
import { createBrowserClient } from "@supabase/ssr";

const supabaseUrl = process.env.NEXT_PUBLIC_SUPABASE_URL || "https://placeholder.supabase.co";
const supabaseKey = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY || "...placeholder...";

export function isSupabaseConfigured(): boolean {
  const url = process.env.NEXT_PUBLIC_SUPABASE_URL || "";
  const key = process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY || "";
  return !!(url && key && !url.includes("placeholder") && !key.includes("placeholder"));
}

export function createClient() {
  return createBrowserClient(supabaseUrl, supabaseKey);
}
```

frontend/src/lib/sounds.ts TypeScript

```typescript
/**
 * Sistema de sonidos para alertas criticas -- Mi Director Financiero PTY
 * Usa Web Audio API (sin archivos MP3, sin dependencias externas).
 */

const SOUND_ENABLED_KEY = "midf_sound_enabled";

export function isSoundEnabled(): boolean {
  if (typeof window === "undefined") return false;
  return localStorage.getItem(SOUND_ENABLED_KEY) !== "false";
}

export function playAlertSound(type: "danger" | "warning" | "reminder"): void {
  if (!isSoundEnabled()) return;
  try {
    const ctx = new (window.AudioContext || window.webkitAudioContext)();
    const oscillator = ctx.createOscillator();
    const gain = ctx.createGain();
    oscillator.connect(gain);
    gain.connect(ctx.destination);

    switch (type) {
      case "danger":  oscillator.frequency.value = 440; gain.gain.value = 0.3; break;
      case "warning": oscillator.frequency.value = 523; gain.gain.value = 0.15; break;
      case "reminder": oscillator.frequency.value = 659; gain.gain.value = 0.1; break;
    }
    oscillator.type = "sine";
    oscillator.start();
    const duration = type === "reminder" ? 0.1 : 0.15;
    gain.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + duration);
    oscillator.stop(ctx.currentTime + duration + 0.05);
    setTimeout(() => ctx.close().catch(() => {}), 500);
  } catch { /* Web Audio API no disponible */ }
}
```

3.7 Componentes de Interfaz de Usuario

frontend/src/components/MidfLogo.tsx TypeScript/React (SVG)

```
"use client";
import React from "react";

/**
 * Mi Director Financiero PTY -- Logo Component
 * Brujula dorada con simbolo $ sobre fondo azul marino (#1A242F)
 * Colores: Dorado #C5A059, Fondo #1A242F
 */
export default function MidfLogo({ size = 40, iconOnly = true }: MidfLogoProps) {
  if (iconOnly) {
    return (
      <svg width={size} height={size} viewBox="0 0 100 100" fill="none">
        <circle cx="50" cy="50" r="50" fill="#1A242F" />
        <circle cx="50" cy="50" r="32" stroke="#C5A059" strokeWidth="1.5" fill="none" />
        {/* Compass cardinal points */}
        <polygon points="50,16 46,38 50,34 54,38" fill="#C5A059" />
        <polygon points="50,84 46,62 50,66 54,62" fill="#C5A059" />
        <polygon points="84,50 62,46 66,50 62,54" fill="#C5A059" />
        <polygon points="16,50 38,46 34,50 38,54" fill="#C5A059" />
        {/* Trend arrow */}
        <line x1="56" y1="38" x2="70" y2="22" stroke="#C5A059" strokeWidth="2.5" />
        {/* Center $ */}
        <circle cx="50" cy="50" r="11" fill="#1A242F" stroke="#C5A059" strokeWidth="2" />
        <text x="50" y="55" textAnchor="middle" fill="#C5A059" fontSize="14"
              fontWeight="bold" fontFamily="serif">$</text>
      </svg>
    );
  }
  // Full logo with text: "MI DIRECTOR FINANCIERO PTY - Tu Aliado Estrategico"
  return (<div className="flex items-center gap-3">...</div>);
}
```

frontend/src/components/AlertsSidebar.tsx TypeScript/React (~332 lineas)

```
"use client";
/**
 * Panel lateral de alertas estrategicas.
 * Agrupa alertas por categoria (DGI, Liquidez, Rentabilidad, RRHH, Inventario)
 * con badges de prioridad y expansion interactiva.
 */
import React, { useState } from "react";
import { Bell, X, AlertTriangle, CheckCircle2, Users, FileText, Package } from "lucide-react";
import type { StrategicAlert, AlertCategory, AlertPriority } from "@/lib/alerts";

const CATEGORY_ICONS: Record<AlertCategory, React.ReactNode> = {
  dgi: <FileText size={14} />,
  capital_humano: <Users size={14} />,
  liquidez: <Droplets size={14} />,
  rentabilidad: <TrendingDown size={14} />,
  legal: <FileText size={14} />,
  inventario: <Package size={14} />,
};

export default function AlertsSidebar({ alerts, isOpen, onClose }: AlertsSidebarProps) {
  const [expandedId, setExpandedId] = useState<string | null>(null);
```

```
  // Agrupa por categoria, muestra badges de prioridad (red/orange/yellow/green)
  // Expansion click-to-reveal con mensaje detallado
  // Boton de campana con contador animado
  // ... (~332 lineas totales)
}

export function AlertBellButton({ alertCount, hasRed, onClick }) {
  // Campana con badge rojo animado (bounce) cuando hay alertas criticas
}
```

3.8 Calculadora de Nomina Panama

frontend/src/components/PayrollEngine.tsx TypeScript/React (~1,148 lineas)

```
"use client";
/**
 * Motor de Calculo de Nomina Panama 2026
 * Ley 462 de 2025 -- CSS Patronal 13.25%
 * Calcula: SS Patronal, SE, Riesgos Prof., XIII Mes, Vacaciones,
 *          Prima Antiguedad, ISR progresivo, retenciones.
 */

export interface PayrollTotals {
  totalGross: number;
  totalEmployerCost: number;
  totalNet: number;
  hiddenCost: number;
  planillaCount: number;
  freelanceCount: number;
  costoAnual: number;
}

function calcularISR(salario: number): number {
  if (salario <= 846.15) return 0;
  if (salario <= 1538.46) return (salario - 846.15) * 0.15;
  let isr = (1538.46 - 846.15) * 0.15;
  isr += (salario - 1538.46) * 0.25;
  return isr;
}

function calcularCargaLocal(salario: number, tipo: string, years: number) {
  if (tipo === "planilla") {
    const ssP = salario * 0.1325;     // Ley 462/2025
    const seP = salario * 0.015;
    const rpP = salario * 0.015;
    const decimo = salario / 12;      // XIII Mes (8.33%)
    const vac = salario * 0.0909;     // 30 dias/11 meses
    const cesantia = salario * 0.0225; // Fondo Cesantia
    const prima = years > 0 ? (salario / 4) / 12 : 0;
    const carga = ssP + seP + rpP + decimo + vac + cesantia + prima;

    const ssE = salario * 0.0975;     // CSS empleado
    const seE = salario * 0.0125;     // SE empleado
    const isrE = calcularISR(salario);
    const totalDed = ssE + seE + isrE;

    return {
      employerCost: salario + carga,
      employeeNet: salario - totalDed,
      totalDeductions: totalDed,
      cargaPatronalPct: (carga / salario) * 100,
      breakdown: {
        "CSS Patronal (13.25%)": ssP,
        "Seguro Educativo (1.50%)": seP,
        "Riesgos Prof. (1.50%)": rpP,
        "Fondo Cesantia (2.25%)": cesantia,
        "Prov. XIII Mes (8.33%)": decimo,
        "Prov. Vacaciones (9.09%)": vac,
        "Prima Antiguedad": prima,
        // ...
      },
```

```
    };
  }
  // Freelance: retencion ISR 10%
  return { employerCost: salario, employeeNet: salario * 0.9, ... };
}

// localStorage persistence (patron identico a LabPrecios)
const EMPLOYEES_KEY = "midf_employees";
function loadEmployeesFromStorage(): Employee[] { ... }
function saveEmployeesToStorage(emps: Employee[]): void { ... }

export default function PayrollEngine({ societyId, onPayrollUpdate }) {
  // ... 1,148 lineas: CRUD empleados, graficos recharts,
  //     desglose carga patronal, XIII Mes, asistencia, exportar CSV
}
```

3.9 Backend — API FastAPI (Python)

backend/main.py Python

```python
"""
Mi Director Financiero PTY - FastAPI Backend
Motor financiero + Capa de Lenguaje Natural + Session Logging
"""
import os
from fastapi import FastAPI, Depends, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from contextlib import asynccontextmanager

from app.database import get_supabase
from app.middleware import RateLimitMiddleware, get_cors_origins
from app.routers import financial, societies, nlp, audit, ai_agents, payroll, accounting, reports, budget

@asynccontextmanager
async def lifespan(app: FastAPI):
    print("Mi Director Financiero PTY API iniciando...")
    yield
    print("Mi Director Financiero PTY API cerrando...")

app = FastAPI(
    title="Mi Director Financiero PTY API",
    description="Motor financiero y legal para emprendedores en Panama",
    version="0.1.0",
    lifespan=lifespan,
)

app.add_middleware(RateLimitMiddleware, requests_per_minute=60)
app.add_middleware(
    CORSMiddleware,
    allow_origins=get_cors_origins(),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(societies.router, prefix="/api/societies", tags=["Sociedades"])
app.include_router(financial.router, prefix="/api/financial", tags=["Motor Financiero"])
app.include_router(nlp.router, prefix="/api/nlp", tags=["Lenguaje Natural"])
app.include_router(audit.router, prefix="/api/audit", tags=["Auditoria"])
app.include_router(ai_agents.router, prefix="/api/ai", tags=["Inteligencia Artificial"])
app.include_router(payroll.router, prefix="/api/payroll", tags=["Nomina"])
app.include_router(accounting.router, prefix="/api/accounting", tags=["Contabilidad"])
app.include_router(reports.router, prefix="/api/reports", tags=["Reportes PDF"])
app.include_router(budget.router, prefix="/api/budget", tags=["Presupuesto"])

@app.get("/")
async def root():
    return {"status": "ok", "app": "Mi Director Financiero PTY", "version": "0.1.0"}
```

backend/app/auth.py Python

```python
"""
Modulo de Autenticacion -- Mi Director Financiero PTY
Valida JWT de Supabase y extrae identidad del usuario.
Soporta DEMO_MODE para desarrollo local sin Supabase Auth.
```

```
"""
import os
from fastapi import Depends, HTTPException, Request
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import httpx

security = HTTPBearer(auto_error=False)

class AuthenticatedUser:
    def __init__(self, id: str, email: str, role: str = "owner"):
        self.id = id
        self.email = email
        self.role = role

async def get_current_user(
    request: Request,
    credentials: Optional[HTTPAuthorizationCredentials] = Depends(security),
) -> AuthenticatedUser:
    """
    Dependencia FastAPI que extrae y valida el JWT de Supabase.
    En DEMO_MODE: retorna demo user sin token.
    En produccion: valida contra Supabase GoTrue /auth/v1/user.
    """
    if credentials is not None:
        return await _validate_supabase_token(credentials.credentials)

    if _is_demo_mode():
        legacy_id = request.headers.get("x-user-id")
        if legacy_id:
            return AuthenticatedUser(id=legacy_id, email="legacy@midf.local")
        return AuthenticatedUser(id="demo-user-001", email="demo@midf.local")

    raise HTTPException(status_code=401, detail="Token de autenticacion requerido.")

async def _validate_supabase_token(token: str) -> AuthenticatedUser:
    supabase_url = os.environ.get("SUPABASE_URL")
    async with httpx.AsyncClient(timeout=10.0) as client:
        response = await client.get(
            f"{supabase_url}/auth/v1/user",
            headers={"Authorization": f"Bearer {token}", "apikey": anon_key},
        )
    if response.status_code != 200:
        raise HTTPException(status_code=401, detail="Token invalido o expirado.")
    user_data = response.json()
    return AuthenticatedUser(id=user_data["id"], email=user_data.get("email", ""))
```

backend/app/database.py Python

```
"""Conexion a Supabase. Usa service_role key para bypass RLS."""
import os
from functools import lru_cache
from supabase import create_client, Client

@lru_cache()
def get_supabase() -> Client:
    url = os.environ["SUPABASE_URL"]
    key = os.environ["SUPABASE_SERVICE_KEY"]
    return create_client(url, key)
```

backend/app/models.py Python

```python
"""Modelos Pydantic para validacion de datos."""
from pydantic import BaseModel, Field
from typing import Optional
from datetime import date
from enum import Enum

class EntityType(str, Enum):
    SA = "SA"
    SRL = "SRL"
    SE = "SE"

class ContractType(str, Enum):
    PAYROLL = "payroll"
    FREELANCE = "freelance"

class SocietyCreate(BaseModel):
    entity_type: EntityType
    legal_name: str = Field(min_length=3, max_length=200)
    trade_name: Optional[str] = None
    tax_id: Optional[str] = None
    incorporation_date: Optional[date] = None
    fiscal_regime: FiscalRegime = FiscalRegime.GENERAL

class FinancialRecordCreate(BaseModel):
    society_id: str
    period_year: int = Field(ge=2020, le=2030)
    period_month: int = Field(ge=1, le=12)
    revenue: float = Field(ge=0, default=0)
    cogs: float = Field(ge=0, default=0)
    opex_rent: float = Field(ge=0, default=0)
    opex_payroll: float = Field(ge=0, default=0)
    opex_other: float = Field(ge=0, default=0)
    depreciation: float = Field(ge=0, default=0)
    interest_expense: float = Field(ge=0, default=0)
    tax_expense: float = Field(ge=0, default=0)
    cash_balance: float = Field(ge=0, default=0)
    accounts_receivable: float = Field(ge=0, default=0)
    inventory: float = Field(ge=0, default=0)
    accounts_payable: float = Field(ge=0, default=0)
    bank_debt: float = Field(ge=0, default=0)

class PayrollEntryCreate(BaseModel):
    society_id: str
    employee_name: str
    contract_type: ContractType
    gross_salary: float = Field(gt=0)
```

backend/app/engines/financial_engine.py Python (~200 lineas)

```python
"""
Motor de Verdad Financiera
Calcula: Cascada P&L, Ratios, Punto de equilibrio, CCC, Valoracion, Diagnostico.
"""

def calcular_cascada(record: dict) -> dict:
    """Cascada de rentabilidad: de la venta a la utilidad neta."""
    revenue = record.get("revenue", 0)
    cogs = record.get("cogs", 0)
    opex_rent = record.get("opex_rent", 0)
    opex_payroll = record.get("opex_payroll", 0)
    opex_other = record.get("opex_other", 0)

    gross_profit = revenue - cogs
    total_opex = opex_rent + opex_payroll + opex_other
    ebitda = gross_profit - total_opex
    ebit = ebitda - record.get("depreciation", 0)
    ebt = ebit - record.get("interest_expense", 0)
    net_income = ebt - record.get("tax_expense", 0)

    return {
        "revenue": revenue,
        "gross_profit": round(gross_profit, 2),
        "ebitda": round(ebitda, 2),
        "ebit": round(ebit, 2),
        "net_income": round(net_income, 2),
        "waterfall_steps": [
            {"label": "Ventas", "value": revenue, "type": "increase"},
            {"label": "Costo Ventas", "value": -cogs, "type": "decrease"},
            {"label": "= Ut. Bruta", "value": gross_profit, "type": "total"},
            {"label": "OPEX", "value": -total_opex, "type": "decrease"},
            {"label": "= EBITDA", "value": ebitda, "type": "total"},
            # ...
        ],
    }

def calcular_ratios(record: dict) -> dict:
    """Ratios financieros operativos y de solvencia."""
    # Margenes, eficiencia operativa, ratios de liquidez...
    # ... (~80 lineas adicionales)
```

backend/app/engines/panama_payroll.py Python (~150 lineas)

```python
"""
Motor de Calculo de Nomina Panama 2026
Ley 462 de 2025 -- CSS Patronal actualizada al 13.25%
"""

def calcular_isr_mensual(salario: float) -> float:
    """
    Tabla ISR Mensual DGI Panama 2026 (3 tramos progresivos).
    Tramo 1: $0 - $846.15 -> 0%
    Tramo 2: $846.16 - $1,538.46 -> 15%
    Tramo 3: >$1,538.46 -> 25%
    """
    if salario <= 846.15:
        return 0.0
    if salario <= 1538.46:
```

```
        return (salario - 846.15) * 0.15
    isr = (1538.46 - 846.15) * 0.15
    isr += (salario - 1538.46) * 0.25
    return isr

def calcular_carga_panama(salario: float, tipo: str, years_worked: int = 0) -> dict:
    """
    Calcula el Costo Real para la empresa y el Neto para el empleado.
    Ley 462 de 2025: CSS Patronal sube de 12.25% a 13.25%.
    """
    if tipo == "payroll":
        ss_patronal = salario * 0.1325
        se_patronal = salario * 0.0150
        rp_patronal = salario * 0.0150
        decimo_prov = salario / 12
        vacaciones_prov = (salario / 30) * 2.5 / 12
        prima_antig = (salario / 4) / 12 if years_worked > 0 else 0.0

        carga_patronal = (ss_patronal + se_patronal + rp_patronal +
                          decimo_prov + vacaciones_prov + prima_antig)
        employer_cost = salario + carga_patronal

        ss_empleado = salario * 0.0975
        se_empleado = salario * 0.0125
        isr_empleado = calcular_isr_mensual(salario)
        total_deductions = ss_empleado + se_empleado + isr_empleado
        employee_net = salario - total_deductions

        return {
            "employer_cost": round(employer_cost, 2),
            "employee_net": round(employee_net, 2),
            "total_deductions": round(total_deductions, 2),
            "breakdown": { ... },
        }
    # Freelance: retencion ISR 10%
    return { "employer_cost": salario, "employee_net": salario * 0.9, ... }
```

3.10 Recursos Humanos — Tipos y CRUD

frontend/src/lib/rrhh-types.ts TypeScript (~600 lineas)

```typescript
/**
 * Mi RRHH -- Types and localStorage utilities
 * Single Source of Truth for employee/freelancer data
 * Feeds: Form 03, F2 V10, Form 20, Planilla CSS
 */

export type TipoPersonal = "empleado" | "freelancer";
export type TipoDocumento = "cedula" | "pasaporte" | "carnet_residente";
export type TipoContrato = "indefinido" | "definido" | "por_obra";
export type FormaPago = "mensual" | "quincenal" | "semanal";
export type EstadoPersonal = "activo" | "inactivo";

export interface PersonalRecord {
  id: string;
  tipo_personal: TipoPersonal;
  tipo_documento: TipoDocumento;
  numero_documento: string;
  dv: string;
  nombre_completo: string;
  telefono: string;
  email: string;
  estado: EstadoPersonal;
  // Empleado fields
  fecha_ingreso: string;
  tipo_contrato: TipoContrato;
  cargo: string;
  salario_mensual: number;
  forma_pago: FormaPago;
  numero_css: string;
  aplica_deduccion_conyugal: boolean;
  deduccion_hipotecaria_anual: number;
  // Freelancer fields
  tipo_servicio: string;
  tarifa_honorario: number;
  modalidad_pago_freelance: ModalidadPagoFreelance;
  retiene_isr: boolean;
}

export interface PlanillaMensualEntry {
  personal_id: string;
  nombre: string;
  salario_base: number;
  dias_trabajados: number;
  horas_extras: number;
  comisiones: number;
  decimo_tercer_mes: number;
  // Calculated
  total_devengado: number;
  css_trabajador: number;
  se_trabajador: number;
  retencion_isr: number;
  total_deducciones: number;
  css_empleador: number;
  se_empleador: number;
  salario_neto: number;
  costo_total_empresa: number;
}
```

```
// ... CRUD localStorage, acumulados anuales, calculo ISR personalizado,
//     planillas mensuales, pagos a freelancers (~600 lineas totales)
```

3.11 Espejo de Formularios DGI

frontend/src/lib/dgi-mappings.ts TypeScript (~450 lineas)

```
/**
 * DGI Form Mappings -- Espejo de Formularios
 * Maps DEFAULT_PANAMA_CHART accounts to official DGI form fields.
 * Form 430: Declaracion Jurada del ITBMS (trimestral)
 * Form 03:  Declaracion Jurada de Renta Anual para S.E.
 * Casilla numbers match the forms shown in e-Tax 2.0 (DGI Panama).
 */

export interface DGIFormField {
  casilla: string;
  label: string;
  accountCodes: string[];
  aggregation: "sum_debe" | "sum_haber" | "net" | "formula";
  formula?: (values: Record<string, number>) => number;
  section: string;
  helpText?: string;
}

export const FORM_430: DGIFormDefinition = {
  formId: "430",
  formName: "Declaracion Jurada del ITBMS",
  frequency: "quarterly",
  dueDateRule: "15 del mes siguiente al cierre del trimestre",
  fields: [
    { casilla: "10", label: "Ingresos Gravados por Ventas y Servicios",
      accountCodes: ["4.1", "4.2"], aggregation: "sum_haber",
      section: "Ingresos Gravados" },
    { casilla: "30", label: "Compras y Gastos Gravados con ITBMS",
      accountCodes: ["5.1.01", "5.1.02", "5.2.05", ...],
      aggregation: "sum_debe", section: "Compras y Gastos" },
    // ... 20+ casillas adicionales
  ],
};

export function computeFormValues(
  form: DGIFormDefinition,
  balances: AccountBalance[]
): ComputedFormResult { ... }

export function getDGICalendar(year: number): DGIDeadline[] { ... }
```

3.12 Componentes Principales (Referencias)

Los siguientes componentes completan la funcionalidad de la aplicacion. Se incluyen sus firmas y descripciones por razon de espacio:

Archivo	Lineas	Descripcion
dashboard/page.tsx	~2,093	Dashboard principal: 5 modulos (Flash, Contabilidad, Espejo DGI, Mi RRHH, Mi Inventario), navegacion, graficos, alertas, simulador, presupuesto, valoracion, Mi Asistente IA
LabPrecios.tsx	~900	Laboratorio de precios con costeo unitario avanzado, conversiones de unidades, ingredientes, margen y ITBMS
EspejoDGI.tsx	~800	Espejo de formularios fiscales: F2 V10, Form 03, Form 430, Form 20, Planilla CSS, Calendario DGI
inventario/MiInventario.tsx	~1,100	Gestion de inventario: catalogo CRUD, 9 tipos de movimiento, dashboard con graficos recharts, alertas ROP
SmartTooltip.tsx	~565	Glosario inteligente: 100+ terminos financieros y legales panamenos con definiciones contextuales
DataEntryWizard.tsx	~550	Wizard de entrada de datos financieros con validacion y formulas automaticas
NaturalLanguageInput.tsx	~350	Input conversacional con GPT-4o para consultas financieras en lenguaje natural
LibroContable.tsx	~450	Libro Diario y Libro Mayor con asientos de partida doble
ChartOfAccounts.tsx	~380	Plan de Cuentas Panama con catalogo estandar
Ley186Monitor.tsx	~300	Monitor de topes Ley 186 (S.E.P.) con cuenta regresiva y alertas
BudgetModule.tsx	~400	Modulo de presupuesto vs real con varianzas
F2V10BorradorScreen.tsx	~500	Borrador Form F2 V10 para Sociedad de Emprendimiento
Form20Screen.tsx	~400	Formulario 20 — Declaracion de pagos a terceros
PlanillaCSSScreen.tsx	~380	Planilla CSS con calculo obrero-patronal

4. Declaracion de Autenticidad

DECLARACION JURADA

Declaro bajo juramento que el codigo fuente presentado en este documento corresponde integramente a la obra denominada **"Mi Director Financiero PTY"**, version 0.1.0, de mi autoria original. La obra consta de **112 archivos fuente** que totalizan aproximadamente **46,917 lineas de codigo**, escritas en los lenguajes TypeScript, JavaScript, Python y CSS.

Los fragmentos aqui incluidos son representativos del conjunto total de la obra y han sido seleccionados para demostrar la originalidad, estructura y complejidad del programa. El codigo fuente completo se encuentra disponible para verificacion ante la Direccion Nacional de Derecho de Autor.

Panama, Marzo de 2026

Soraya Dayan Gutierrez Perez

Autora y Titular